

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. CSP-J/S第二轮考试测评机所用的 Linux系统属于（ ）。
A. UML
B. IDE
C. OS
D. Database
2. C++ 中被 `static` 修饰的变量会存储在（ ）中。
A. 堆
B. 栈
C. 静态存储区
D. 寄存器
3. 下列IP中（ ）属于**B类IP**。
A. 8.8.8.8
B. 63.31.255.255
C. 173.30.250.1
D. 193.168.110.120
4. 8 位有符号整型 **-97** 的补码表示是（ ）。
A. 00011110
B. 10011111
C. 10011110
D. 11100001
5. 下列代码中（ ）无法实现 `swap(a, b)`，即两个正整数a、b的交换。
A. `a ^= b ^= a ^= b`
B. `a -= b -= a += b`
C. `a /= b = (a *= b) / b`
D. `t = a, a = b, b = t`
6. 最大长度为**30**的循环队列中，头指针指向无效元素，尾指针指向有效元素，则当 `head = 23` 且 `tail = 15` 时，队列中的元素数为（ ）。
A. 8
B. 9
C. 22
D. 23
7. 5 个人排成一排，甲乙相邻，丙丁不相邻的方案数为（ ）。
A. 24
B. 48

- C. 12
- D. 60
8. 10 个人去买价值5元的商品，其中 5 人拿着 5 元，剩下 5 人拿着 10 元，小卖部开始没有钱可以找，有（ ）种排队方式能使得每个人都找的开。
- A. 14
- B. 42
- C. 24
- D. 28
9. 一棵二叉树的前序遍历结果为 CDBGHAFIE，中序遍历结果为 BDGHCFAIE，该二叉树的后序遍历结果是（ ）。
- A. BHGDFEIIAC
- B. CAIEFDGHB
- C. EIFAHGBDC
- D. BGHDFAIIEC
10. 已知2024年8月29日是星期四，则1842年8月29日是（ ）。
- A. 星期一
- B. 星期三
- C. 星期五
- D. 星期日
11. 某单位安装一条电信宽带进行上网，运营商说下行速度是 1000 Mbps。要下载大小为 20 GB 的软件，最快大约需要（ ）秒。
- A. 2
- B. 20
- C. 200
- D. 2000
12. 如果字符串s在字符串A中出现了，则字符串s被称作字符串A的子串。设字符串 A = "players"，A 的非空子串的数目是（ ）。
- A. 21
- B. 22
- C. 28
- D. 29
13. 在数组 A[x] 中，若存在 $(i < j)$ 且 $(A[i] > A[j])$ ，则称 $(A[i], A[j])$ 为数组 A[x] 的一个逆序对。对于序列 (7,4,1,9,3,6,8,5)，在不改变顺序的情况下，去掉（ ）会使逆序对的个数减少4。
- A. 1
- B. 3
- C. 6
- D. 5

14. 对数列 {50, 40, 95, 20, 15, 70, 60, 45} 以初始增量值 4 进行希尔排序, 则第二轮结束后数列的前 4 位为 ()。

- A. {15, 20, 50, 40}
- B. {15, 40, 60, 20}
- C. {15, 20, 40, 45}
- D. {15, 20, 40, 50}

15. 有 8 个结点的非连通简单无向图最多有 () 条边。

- A. 8
- B. 7
- C. 21
- D. 49

二、阅读程序 (程序输入不超过数组或字符串定义的范围; 判断题正确填√, 错误填×; 除特殊说明外, 判断题每题 1.5 分, 选择题每题 3 分, 共计 40 分)

(一)

```
1  #include<stdio>
2  using namespace std;
3  const int N = 1e3;
4  int n;
5  int used[N];
6  int dfs(int k) {
7      if (k > n) return 1;
8      int res = 0;
9      for (int i = 1; i <= n; ++i) {
10         if (used[i] && i * i >= i) continue;
11         used[i] = 1;
12         res += dfs(k + 1);
13         used[i] = 0;
14     }
15     return res;
16 }
17 int main() {
18     scanf("%d", &n);
19     printf("%d\n", dfs(1));
20     return 0;
21 }
```

判断题:

16. 当程序输入为 12 时, 程序的运行时间可能会超过 1s。 ()

- A. 正确
- B. 错误

17. 当且仅当输入的 n 满足 $n \geq 3$ 时, 程序的输出结果会大于 n 。 ()

- A. 正确
- B. 错误

18. 将程序第10行中"&& i * i >= i"的内容去掉，程序运行结果可能发生改变。（）

A. 正确

B. 错误

19. 删去程序第 2 行"using namespace std;"，程序能成功通过编译。（）

A. 正确

B. 错误

选择题：

20. 当程序输入为 8 时，程序输出为（）。

A. 15014

B. 1854

C. 133496

D. 40320

21. 当程序输入为 11 时，程序输出为（）。

A. 176214841

B. 1335972

C. 39916800

D. 1334961

(二)

```
1  #include<iostream>
2  using namespace std;
3
4  const int N = 5e3;
5
6  int n, m;
7  int dp[N][N];
8
9  int main() {
10     cin >> n >> m;
11     dp[0][0] = 1;
12     for (int i = 1; i <= m; ++i) {
13         for (int j = 0; j <= n; ++j) {
14             dp[i][j] = dp[i - 1][j];
15             if (j >= i) {
16                 dp[i][j] += dp[i][j - i];
17             }
18         }
19     }
20     cout << dp[m][n] << endl;
21     return 0;
22 }
```

判断题：

22. 程序套用了**0/1背包**模型。 ()

A. 正确

B. 错误

23. 当程序输入为 `5000 1000` 时，程序有 `Runtime error` 的风险。 ()

A. 正确

B. 错误

24. 当程序第 4 行改为 `const int N = 5e6+5;` 后，当输入为 `1000000 500000` 时，程序能在 1s 内正常结束。 ()

A. 正确

B. 错误

25. 当程序输入为 `0 10` 时，程序输出为 `0` 。 ()

A. 正确

B. 错误

选择题：

26. 当程序输入为 `3 0` 时，程序输出为 () 。

A. 3

B. 1

C. 0

D. 程序无输出

27. 当程序输入为 `10 5` 时，程序输出为 () 。

A. 42

B. 30

C. 18

D. 0

(三)

```
1  #include <iostream>
2  using namespace std;
3
4  int c[100], n, s, cnt, ans;
5
6  void calln(int x) {
7      while (x) {
8          n++;
9          x >>= 1;
10     }
11 }
12
13 int bitcount(int x) {
```

```

14     int cnt = 0;
15     while (x) {
16         cnt += x & 1;
17         x >>= 1;
18     }
19     return cnt;
20 }
21
22 int main() {
23     cin >> s;
24     calln(s);
25     for (int i = s; i; i = s & (i-1)) {
26         cnt++;
27         c[bitcount(i)] += i;
28     }
29     for (int i = 1; i <= n; i++)
30         ans = max(ans, c[i]);
31     cout << cnt << endl;
32     cout << ans << endl;
33     return 0;
34 }

```

假设输入的 s 为正整数且不超过 10^9 ，完成下面的判断题和单选题：

判断题：

28. 输出的第一行不会超过输入的 s 。（）

- A. 正确
- B. 错误

29. $bitcount(x)$ 函数用于计算 x 对应的二进制整数的位数。（）

- A. 正确
- B. 错误

30. 输入的 s 不应大于 100，否则会发生数组越界。（）

- A. 正确
- B. 错误

31. 当程序输入为 3 时，程序的输出的第一行和第二行都是 3。（）

- A. 正确
- B. 错误

选择题：

32. 当程序输入的 s 为 23 时，程序输出的第一行为（）。

- A. 3
- B. 7
- C. 15
- D. 31

33. 当程序输入的 s 为 11 时，程序输出的第二行为（）。

- A. 11
- B. 16
- C. 19
- D. 22

34. 当程序输入的 s 为 127 时，程序输出的第二行为（）【本题4分】。

- A. 1980
- B. 2540
- C. 2870
- D. 3200

三、完善程序（单选题，每小题3分，共计30分）

（一）冒泡排序

题目描述

Mike 有一个数列，他想对这个数列进行排序，但是他只会冒泡排序，于是他想知道，他至少需要交换相邻的数几次，才能将整个数列排成升序呢。

输入格式

第一行一个整数 n ，表示数列的长度。

第二行 n 个整数 $a_1, a_2, \dots, a_n$ ，表示数列。

输出格式

一行，包含一个整数，表示最少交换次数。

提示

冒泡排序交换次数即数列逆序对数，此处用归并排序统计逆序对。

数据规模： $1 \leq n \leq 10^6$ ， $-10^9 \leq a_i \leq 10^9$ 。

程序

```
1  #include<cstdio>
2  #include<cstring>
3
4  typedef long long LL;
5  const int N = 1000010;
6
7  int n;
8  LL ans;
9  int a[N], t[N];
10
11 void sort(int l, int r) {
12     if (l + 1 == r) return;
13     int mid = l + r >> 1;
14     ____①____;
15     for (____②____; k < r; ++k)
16         if (j >= r || (i < mid && a[i] <= a[j])) t[k] = a[i++];
17         else t[k] = a[j++], ____③____;
```

```

18     ____④____;
19 }
20
21 int main() {
22     scanf("%d", &n);
23     for (int i = 0; i < n; ++i) {
24         scanf("%d", a + i);
25     }
26     ____⑤____;
27     printf("%11d\n", ans);
28     return 0;
29 }

```

35. ①处应填 () 。

- A. `sort(l, mid), sort(mid + 1, r)`
- B. `sort(l, mid), sort(mid, r)`
- C. `sort(l, mid - 1), sort(mid, r)`
- D. `sort(l, mid - 1), sort(mid + 1, r)`

36. ②处应填 () 。

- A. `int k = l, i = l, j = mid + 1`
- B. `int k = l, i = mid + 1, j = l`
- C. `int k = l, i = mid + 1, j = r`
- D. `int k = l, i = l, j = mid`

37. ③处应填 () 。

- A. `++ans`
- B. `ans += mid - i + 1`
- C. `ans += mid - i - 1`
- D. `ans += mid - i`

38. ④处应填 () 。

- A. `memcpy(a + l, t + l, sizeof(int) * (r - l))`
- B. `memcpy(a, t, sizeof(a))`
- C. `memset(t, 0, sizeof(t))`
- D. `memcpy(a + l, t + l, sizeof(int) * (r - l + 1))`

39. ⑤处应填 () 。

- A. `sort(1, n)`
- B. `sort(1, n + 1)`
- C. `sort(0, n)`
- D. `sort(0, n + 1)`

(二) 拥挤的城市

题目描述

Tom 所在的城市由 n 个地点组成，并由 $n - 1$ 条公路连接，**这些公路的分布保证地点之间两两有且仅有一条路径。**

Tom 所在的城市十分拥挤，这使 Tom 每次去某地都非常困扰，他为每条公路定义了一个拥挤值，他定义两个地点之间的拥挤程度为两地路径上每条公路的拥挤值的最大值。

现在 Tom 想知道，根据他的定义，任意两两城市间拥挤程度的和是多少呢？

输入格式

第一行一个整数 n ，表示城市中的地点个数。

接下来 n 行每行三个整数 x, y, v 表示在地点 x 和 y 之间有一条拥挤值为 v 的公路。

输出格式

一行，包含一个整数，表示任意两两城市间拥挤程度的和。

提示

将边根据拥挤值排序，分别计算每条边对答案的贡献，用并查集维护连通块大小。

数据规模： $1 \leq n \leq 10^5$ ， $1 \leq x, y \leq n$ ， $1 \leq v \leq 10^7$ 。

程序

```
1  #include<iostream>
2  #include<algorithm>
3  using namespace std;
4
5  typedef long long LL;
6  const int N = 100010;
7
8  struct Edge {
9      int x, y;
10     LL v;
11 };
12 inline bool cmp(Edge A, Edge B) {
13     return ____①____;
14 }
15
16 int n;
17 LL ans;
18 int fa[N], sz[N];
19 Edge e[N];
20
21 int find(int x) {
22     return fa[x] == x ? x : ____②____;
23 }
24
25 int main() {
26     cin >> n;
27     for (int i = 1; i <= n; ++i) {
28         ____③____;
29     }
30     for (int i = 1; i < n; ++i) {
```

```

31     cin >> e[i].x >> e[i].y >> e[i].v;
32     }
33     sort(e + 1, e + n, cmp);
34     for (int i = 1; i < n; ++i) {
35         int x = find(e[i].x), y = find(e[i].y);
36         ans += ____④____;
37         fa[x] = y;
38         ____⑤____;
39     }
40     cout << ans << endl;
41     return 0;
42 }

```

40. ①处应填 () 。

- A. `A.x > B.x`
- B. `A.x < B.x`
- C. `A.v < B.v`
- D. `A.v > B.v`

41. ②处应填 () 。

- A. `find(fa[x])`
- B. `fa[x] = find(x)`
- C. `fa[x] = find(fa[x])`
- D. `x = find(fa[x])`

42. ③处应填 () 。

- A. `fa[i] = i, sz[i] = 1`
- B. `fa[i] = sz[i] = 0`
- C. `fa[i] = 0, sz[i] = 1`
- D. `fa[i] = 0, sz[i] = i`

43. ④处应填 () 。

- A. `sz[x] + sz[y]`
- B. `(sz[x] + sz[y]) * e[i].v`
- C. `sz[x] * sz[y] * e[i].v`
- D. `e[i].v * sz[x] * sz[y]`

44. ⑤处应填 () 。

- A. `sz[x] += sz[y]`
- B. `fa[y] = x`
- C. `sz[x] = 0`
- D. `sz[y] += sz[x]`

